Vectorized Query Execution

Marija Ćetković

October 31, 2025

1 Introduction

Vectorization is the process of converting an algorithm's scalar implementation (which processes single operations at a time) to a vector implementation that processes one operation on multiple pairs of operands simultaneously. This technique is known as **Data Parallelization**.

To highlight the importance of vectorization, in databases, processing tuple by tuple means the work within a tuple is independent of other tuples, ensuring no computational interference. When one tuple is finished, a new one can start, but it must be read from memory and processed.

Potential Speed-up: Suppose the DBMS can parallelize an algorithm over 32 cores, and each core has 4-wide SIMD registers. The potential speed-up is $32x \times 4x = 128x$.

2 Single Instruction, Multiple Data (SIMD)

SIMD refers to a class of CPU instructions that allows the processor to perform the **same operation** on **multiple data points** simultaneously.

SISD (Single Instruction Single Data): Requires a sequential loop processing one element pair at a time.

SIMD: Stores vectors in wide registers (e.g., 128-bit SIMD Registers) and performs the operation concurrently on multiple data points (e.g., four scalar values).

2.0.1 Vectorization Direction

1. **Approach #1: Horizontal:** Perform the operation on all elements together within a single vector.

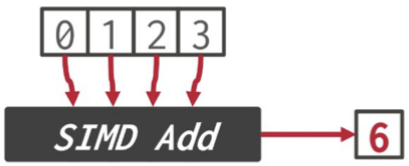

Figure 1: Horizontal Vectorization

2. **Approach #2: Vertical:** Perform the operation element-wise on elements of each vector.

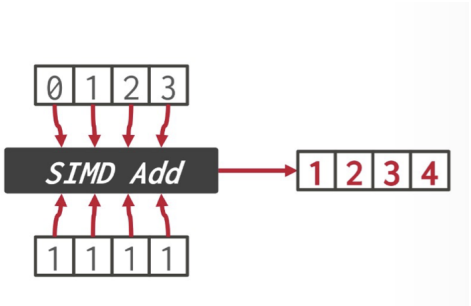

Figure 2: Vertical Vectorization

2.1 AVX-512

AVX-512 is Intel's most advanced SIMD extension, expanding the register width from 256 bits (in AVX2) to 512 bits. This means a single instruction can process eight 64-bit values or sixteen 32-bit values in parallel. Beyond just wider registers, AVX-512 adds new features such as masking (predication), scatter/gather operations, and data permutation, of which make vectorized processing more flexible. It focuses on working on data in registers and performing fast data movement between registers and memory, to achieve speed. Despite its potential for massive parallelism, real-world performance depends on the CPU's ability to decode and deal with wide instructions efficiently, so some bottlenecks can appear in practice.

3 Implementation Approaches

3.1 Automatic Vectorization

The compiler tries to detect when instructions inside a loop can be rewritten as vectorized operations. This usually works only for simple loops and is uncommon in database operators. It requires hardware support for SIMD instructions, and compilers often struggle with nested loops. Automatic vectorization fails if the compiler cannot be sure that arrays do not overlap in memory. If it suspects that two pointers might refer to the same location, it avoids vectorization to prevent incorrect results.

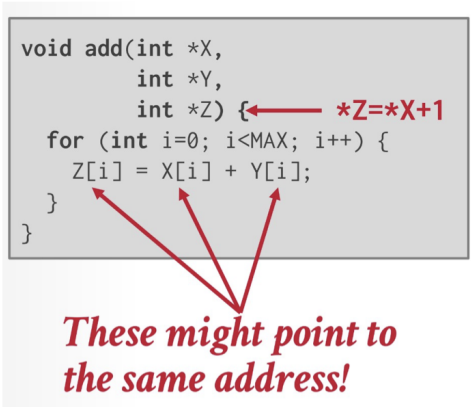

Figure 3: Automatic Vectorization

3.2 Compiler Hints

This approach provides the compiler with additional information to assure it that vectorization is safe. Two primary approaches exist: explicit information about memory or telling the compiler to ignore vector dependencies.

Memory Location Hints (`restrict` keyword): In C/C++, `restrict` tells the compiler that the pointed-to arrays (`X`, `Y`, `Z`) are distinct memory locations for the lifetime of the pointers. The compiler can then safely vectorize operations on that data.

```
void add(int *restrict X,
         int *restrict Y,
         int *restrict Z) {
  for (int i=0; i<MAX; i++) {
    Z[i] = X[i] + Y[i];
  }
}
```

Figure 4: Restrict keyword

Ignoring Vector Dependencies: A pragma tells the compiler to ignore loop dependencies for the vectors. It is up to the DBMS developer to make sure this is the intended behavior.

```
void add(int *X,
         int *Y,
         int *Z) {
#pragma ivdep
  for (int i=0; i<MAX; i++) {
    Z[i] = X[i] + Y[i];
  }
}
```

Figure 5: Pragma example

3.3 Explicit Vectorization

Explicit vectorization is when the programmer directly controls how computations are executed on multiple data elements at once. This can be done either by writing processor-specific code using SIMD intrinsics, which is fast but tied to a particular CPU, or by using vector libraries like Google Highway, Simd, EVE. These libraries allow developers to work with fixed-size vectors (e.g., vectors of size 8 storing 32-bit elements) in a hardware-independent way. The library maps the vector operations to whatever instructions are available on the underlying processor, making the code portable while still taking advantage of SIMD parallelism.

```
void add(int *X,
         int *Y,
         int *Z) {
  __m128i *vecX = (__m128i*)X;
  __m128i *vecY = (__m128i*)Y;
  __m128i *vecZ = (__m128i*)Z;
  for (int i=0; i<MAX/4; i++) {
    _mm_store_si128(vecZ++,
      ↳_mm_add_epi32(*vecX++,
                     ↳*vecY++));
  }
}
```

Figure 6: CPU intrinsic example

3.4 Practical Outcomes of Automatic Vectorization

Compilers such as GCC, Clang, and ICC were tested for their ability to automatically vectorize Vectorwise primitives. ICC performed best, successfully vectorizing operations like Hashing, Selection, and

Projection, while others, such as Hash Table Probing and Aggregation, could not be vectorized automatically [1].

4 Vectorization Fundamentals (SIMD Operations)

The DBMS uses fundamental SIMD operations to build more complex functionality.

4.1 Key Operations

1. **Masking (Predication)**: The CPU performs operations only on lanes specified by an input bitmask. This is used to eliminate values discarded by a selection or filter. Almost all AVX-512 operations support predication variants. Active lanes (1) execute the operation; inactive lanes (0) take values from a "merge source" vector.

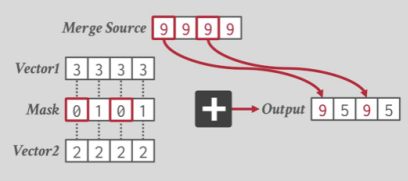

Figure 7: Masking

2. **Permute**: For each lane, copy values from the Input Vector specified by the offset in the Index Vector into the destination vector.

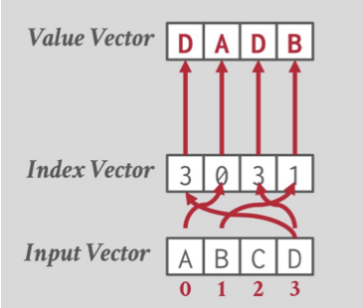

Figure 8: Permute

Prior to AVX-512, achieving permutation often required writing data from the SIMD register to memory and then reading it back into the SIMD register, which is very slow.

3. **Compress/Expand**:

   Compress: Uses an Index Vector to select active elements from the Input Vector and packs them tightly to the front of the destination vector.

   Expand: Takes data from the front of the source vector and places it into specific positions in the destination vector, determined by the mask.

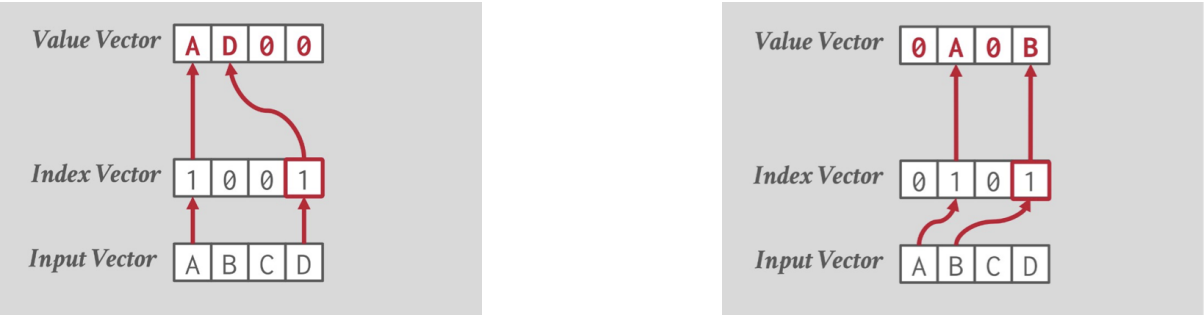

Figure 9: Compress/Expand

4. **Selective Load/Store**:

   Selective Load: Reads data from a contiguous block of memory, loads certain elements into a vector based on a mask.

   Selective Store: Writes data from a vector back to memory based on a mask, sequentially storing the masked (selected) elements.

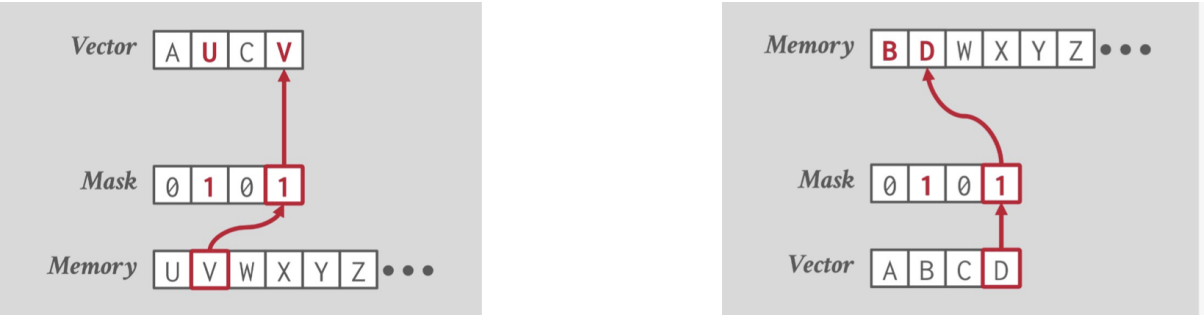

Figure 10: Load/Store

5. **Selective Gather/Scatter**:

   Selective Gather: Reads non-contiguous data from memory into a vector based on an Index Vector (offsets), and permutes the results.

   Selective Scatter: Stores vector elements to non-contiguous locations in memory, permuted by an Index Vector. If collisions occur, the value from the rightmost lane is stored.

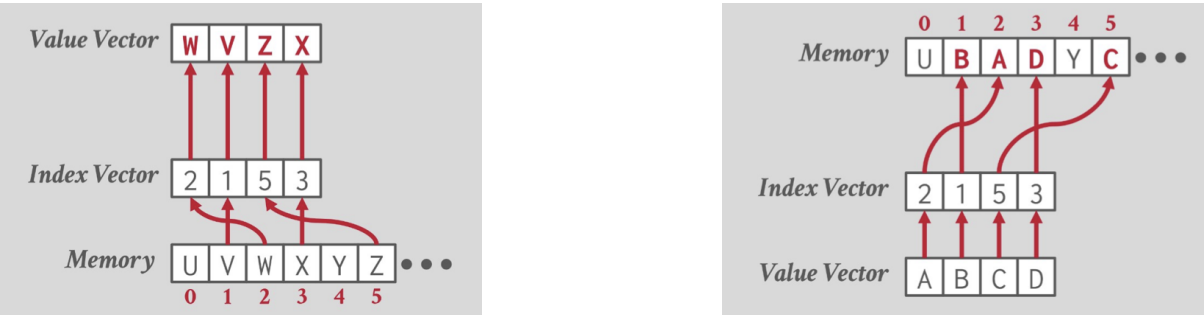

Figure 11: Gather/Scatter

5 Vectorized DBMS Algorithms

In SIMD execution, each logical lane of the vector holds a single element being processed. During a selection, some lanes may become inactive if their element does not satisfy the condition. In a hash join, the process involves building a hash table and then probing it, which can take a different number of steps for each element depending on collisions and matches. Despite these differences, every operator produces output for the next vertical lane, and the same operation is applied across all lanes of data (single instruction), enabling parallel processing while keeping track the individual status (active or inactive) of each element.

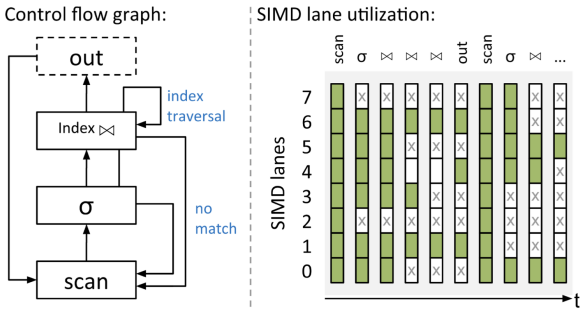

Figure 12: SIMD Lanes [2]

Principles for Efficient Vectorization [2]

- Favor **vertical vectorization** by processing different input data per lane.
- **Maximize lane utilization** by executing unique data items per lane subset (avoiding useless computations or idle lanes).

5.1 Selection Scans

A selection scan filters table rows based on certain conditions. In the vectorized version, operation starts by loading a vector of keys from the table into SIMD registers. Then, two comparison masks are created — one marking elements greater than or equal to N, and another for elements less than or equal to U. These masks are combined with a SIMD AND operation to identify which tuples meet both criteria. Finally, only the qualifying tuples, as indicated by the resulting mask, are written to the output using a SIMD store operation.

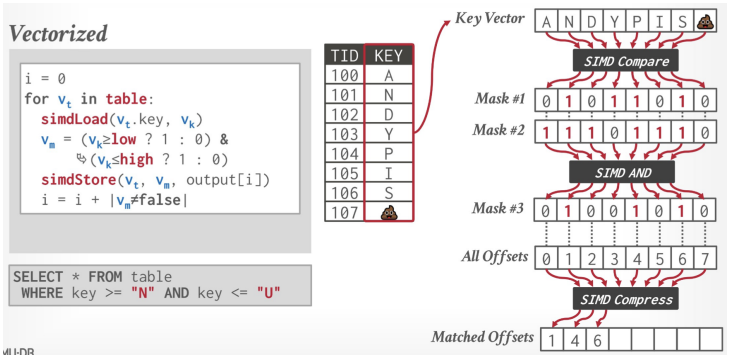

Figure 13: Selection scan

5.2 Relaxed Operator Fusion (ROF)

Relaxed Operator Fusion (ROF) is a vectorized processing model used in query compilation engines.[3] Its main goal is to reduce wasted work when tuples are filtered out but still processed by later operators. ROF splits the query pipeline into smaller **stages**, each operating on a vector of tuples. Between stages, data

passes through small **cache-friendly buffers**, which both remove filtered-out tuples and set boundaries for vectorization. This helps keep the CPU cache efficient and avoids unnecessary computation. ROF also supports **prefetching**, meaning the CPU can start loading the next batch of data while still working on the current one. This reduces memory stalls and keeps SIMD lanes active. Common prefetching methods include group prefetching for sequential data, software pipelining to overlap memory and computation, and AMAC for adaptive access patterns.

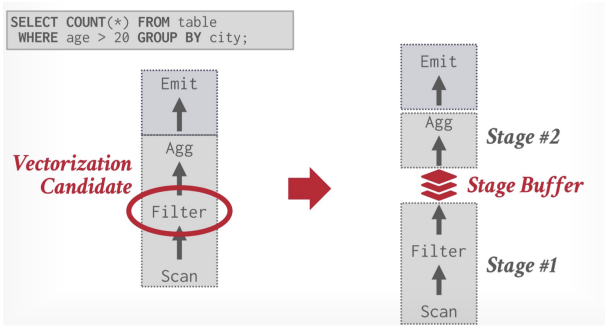

Figure 14: ROF Example

5.3 Vector Refill Algorithms

When processing data in batches, some SIMD lanes may become empty if certain tuples are filtered out. To keep all lanes busy, **vector refill strategies** can be used.[2]

1. **Buffered refill:** The operator temporarily stores valid tuples in extra buffer. Once there is enough data, it fills the empty lanes in the next iteration from the buffer and new data combined.

2. **Partial refill:** Instead of waiting for the next iteration, the operator keeps the partially filled vector and requests additional tuples from the previous operator to fill the remaining lanes. This requires careful coordination to ensure that these saved vectors are not accidentally overwritten by other operations.

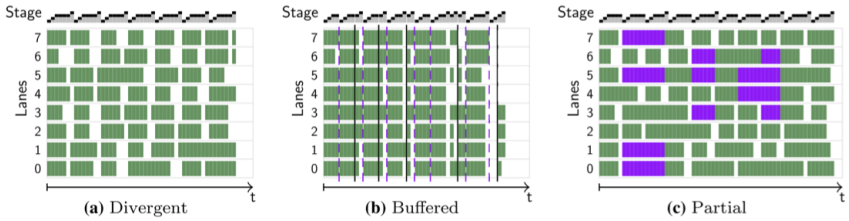

Figure 15: Different refill algorithms comparison [2]

5.4 Hash Tables

5.4.1 Scalar Linear Probing

In scalar probing, the algorithm processes one input key at a time. For each key (`k1`), it computes a hash index (`h1`) and checks the corresponding position in the hash table. If the key is not found at that index, the algorithm continues probing the next slot linearly (by incrementing) until it either finds a matching key or reaches an empty entry, which means that the key is not present.

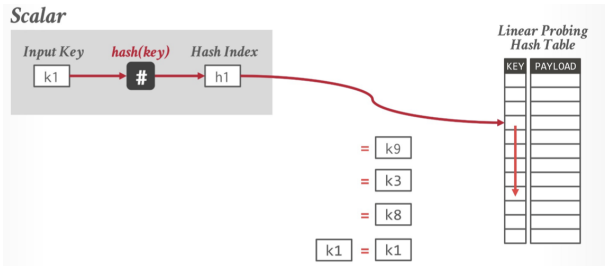

Figure 16: Scalar probing

5.4.2 Vectorized Probing (Horizontal)

Horizontal vectorized probing is used with bucketized hash tables, where each bucket contains multiple key–value pairs (for example, four keys and four values). Instead of checking just one entry per probe, the algorithm takes a single input key (`k1`), computes its hash (`h1`), and then compares this key against all the keys in the target bucket simultaneously using a SIMD comparison. The result is a "matched mask," such as $(0, 0, 0, 1)$, where 1 is the position of the matched value within the bucket.

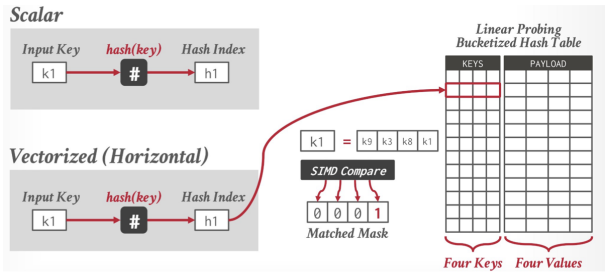

Figure 17: Horizontal vectorized probing

5.4.3 Vectorized Probing (Vertical)

In vertical vectorized probing, the algorithm processes multiple input keys in parallel. A vector of input keys (`k1` - `k4`) is first hashed to produce a vector of hash indices (`h1` - `h4`). Using SIMD gather instructions, the algorithm retrieves the keys stored at those indices in a single operation. A SIMD comparison is then performed to check for matches across all lanes. Keys that do not find a match (misses) increment their hash indices (`h2+1`, `h3+1`) and are retried in the next probe iteration, while keys that successfully match (`k1` and `k4`) are replaced by new input elements (`k5` and `k6`) in the vector.

Figure 18: Vertical vectorized probing

5.5 Partitioning / Histograms

When building a histogram using SIMD, each lane in a vector processes a different element. Sometimes, multiple lanes want to increment the same histogram bin at the same time, which can cause conflicts. To avoid this, the algorithm creates a separate copy of the histogram for each lane. Each lane updates

its own copy using scatter and add operations, which safely apply the changes in parallel. After all lanes are done, the copies are combined by summing the counts in each bin, producing the final histogram.

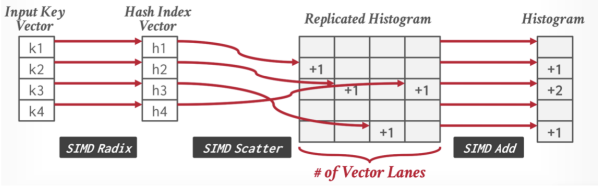

Figure 19: Histograms

6 AVX-512 Discussion

AVX-512 is not always faster than AVX2. While it can process twice as many data elements per instruction, real-world performance depends on how the CPU handles these wide instructions. Some CPUs reduce their clockspeed when running AVX-512 to manage power and heat, which can offset the potential speedup. For this reason, compilers often prefer AVX2 (256-bit SIMD) instructions, since they typically maintain higher CPU frequencies and can achieve better overall throughput. Additionally, if only a small part of a query uses AVX-512, the clock reduction may slow down the entire query, making AVX-512 less efficient in mixed workloads.

7 Conclusion

Vectorization is essential for OLAP. It improves CPU usage and reduces overhead by processing many tuples at once.

SIMD-aware programming is mostly manual. Developers need to adjust data layouts and control flow, as compilers rarely vectorize automatically.

We can combine parallelism. Modern databases combine multi-threading, compiled query plans, and vectorized operators to use CPUs efficiently.

Algorithm vs Hardware Improving OLAP performance can focus on either algorithms or hardware. Ideally, we want both: design algorithms to use current hardware efficiently, and let hardware advances enable more parallelism and vectorization.

References

[1] Timo Kersten, Viktor Leis, Alfons Kemper, Thomas Neumann, Andrew Pavlo, and Peter Boncz. Everything you always wanted to know about compiled and vectorized queries but were afraid to ask. *Proceedings of the VLDB Endowment*, 11(13):2209–2222, 2018.

[2] Harald Lang, Andreas Kipf, Linnea Passing, Peter Boncz, Thomas Neumann, and Alfons Kemper. Make the most out of your simd investments: Counter control flow divergence in compiled query pipelines. In *14th International Workshop on Data Management on New Hardware (DaMoN 2018)*, 2018.

[3] Prashanth Menon, Todd C. Mowry, and Andrew Pavlo. Relaxed operator fusion for in-memory databases: Making compilation, vectorization, and prefetching work together at last. *Proceedings of the VLDB Endowment*, 11(1):1–13, 2017.